פרויקט גמר רשתות תקשורת - FAIRSYNC

מגישים:
איתמר כהן אהרוני ת.ז: 329124556
רפאל בן לולו ת.ז: 209971191
רון אלגזאר ת.ז: 216307926

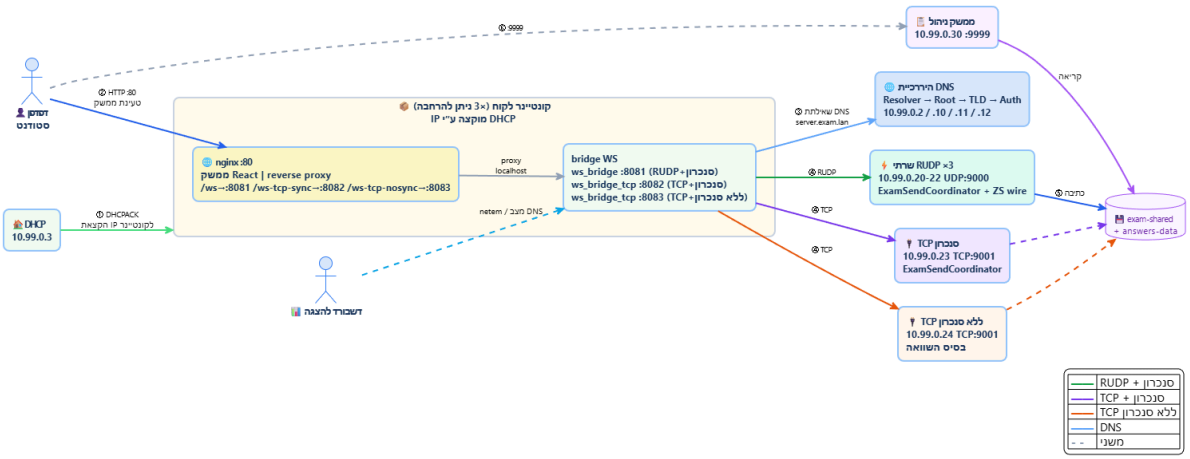
מטרת הפרויקט

אפליקציה לשליחת תוכן (במימוש שלנו קובץ JSON) כך שכל המשתמשים יקבלו את כל אחת מהחבילות באותו הזמן (קירוב כמה שניתן) וגם סנכרון חבילות שבזמן נתון כל אחד מהמשתמשים יקבל את אותן חבילות כמו שאר המשתמשים (+- חבילה בודדת).

הפרויקט יוצר מערכת לשליחת קבצים בצורה שתהיה הוגנת לכל אחד מהמשתמשים ללא "ענישה" למשתמש עם אינטרנט פחות טוב.

יש כמה שימושים שונים למערכת הדורשים התאמה קטנה מצד המתכנת למעבר בינהם לדוג' סימולציה הדורשת כמה מחשבים או רכיבים , מבחן הדורש קבלה של המבחן באותו הזמן לכל המשתמשים, סימולצית מסחר של שוק ההון הדורשת שכל מחשב המדמה סוחר יקבל את אותו המידע באותו הזמן, בנצמרק לכמה מחשבים בו זמנית ועוד הרבה דוגמאות.

בפרוייקט הזה אנחנו שולחים שאלות (קובץ JSON) למערכת שיודעת להציג אותן. לאחר השוואה לאלגוריתם אחר שעושה דבר דומה שלנו יצא מהיר יותר ויעיל יותר.



כפי שמתואר בתרשים המאוד כללי, הפרויקט בנוי בארכיטקטורה מרובת שכבות הרצה כולה בתוך Docker. הסטודנט מתחבר לממשק המשתמש דרך HTTP:80, כאשר קונטיינר הלקוח שכתובת ה-IP שלו הוקצתה דינמית על-ידי שרת הDHCP. הקונטיינר מריץ בתוכו nginx כreverse proxy המנתב בין שלושה bridge processes: ws_bridge לתקשורת RUDP עם סינכרון (port 8081), ו-ws_bridge_tcp בשני מודים TCP עם סינכרון (port 8082) וללא סינכרון (port 8083). לפני כל חיבור, הbridge מבצע שאילתת DNS לגילוי שרת הבחינה, המעוברת דרך היררכיית הDNS המלאה (אפשר גם רקורסיבית וגם איטראטיבית). בצד השרת פועלים שלושה מופעי 10.99.0.20-22) RUDP) ושני מופעי 10.99.0.23/24) TCP) ניתן להגדיל ולהרחיב את השרתים במידת הצורך והround robin יחלק בהתאם, כאשר כולם כותבים לvolumes משותפים (exam-shared ו-answers-data). דשבורד הניהול (10.99.0.30:9999) קורא משני הvolumes ומספק תצוגת מצב בזמן אמת, ודשבורד הפרופסור מאפשר שליטה בתנאי הרשת באמצעות הזרקת עיכובים דרך netem.

רכיבי המערכת

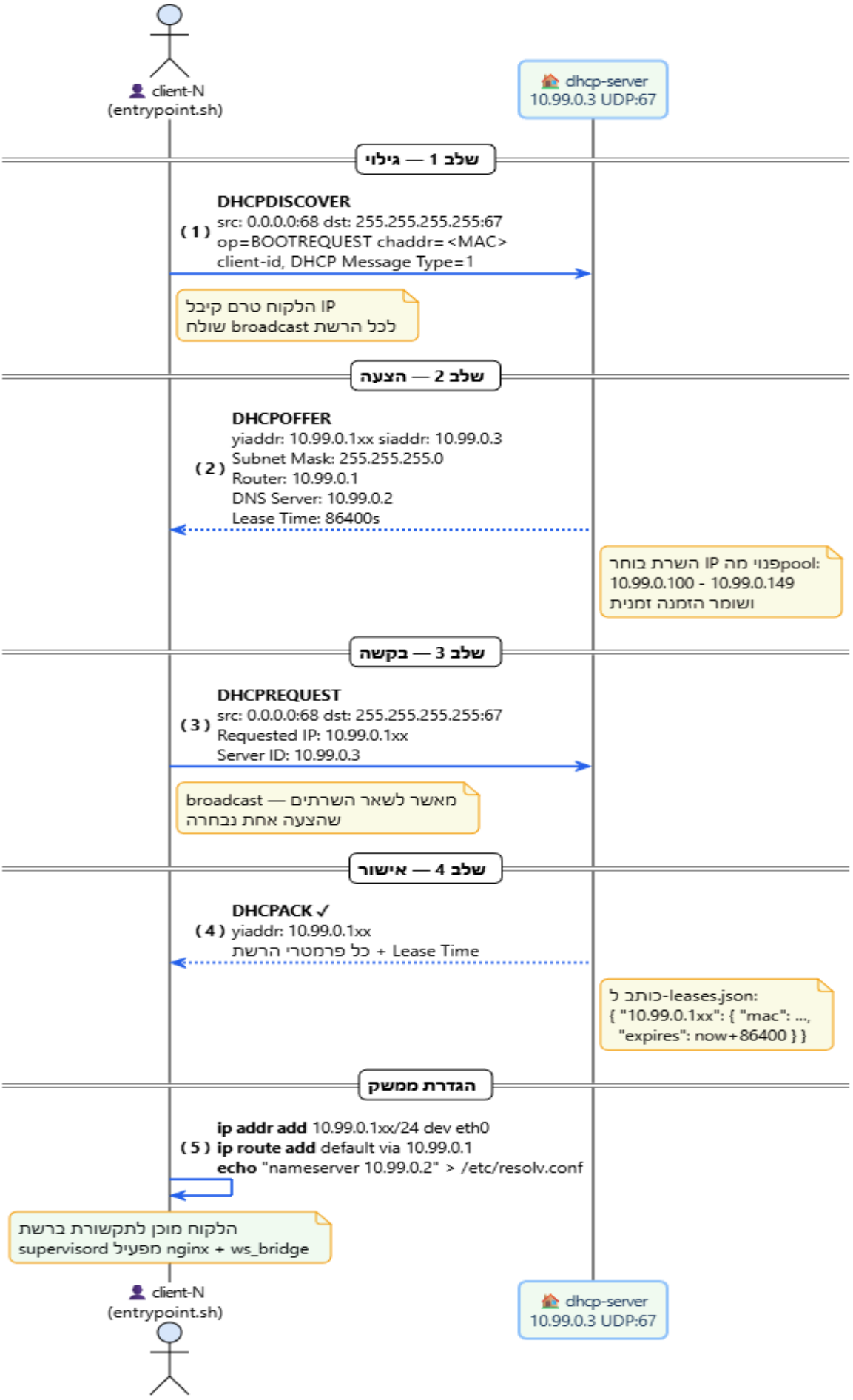
1. DHCP Server

שרת ה-DHCP אחראי להקצות כתובת IP ללקוח בעת ההתחברות לרשת. הוא מקבל כתובת MAC מהמשתמש ומחזיר כתובת IP שתוקצה לזמן מסויים אחרי הדיון (DORA).

תרשים מהפרויקט:

DHCP — רכישת כתובת IP

DISCOVER → OFFER → REQUEST → ACK | RFC 2131

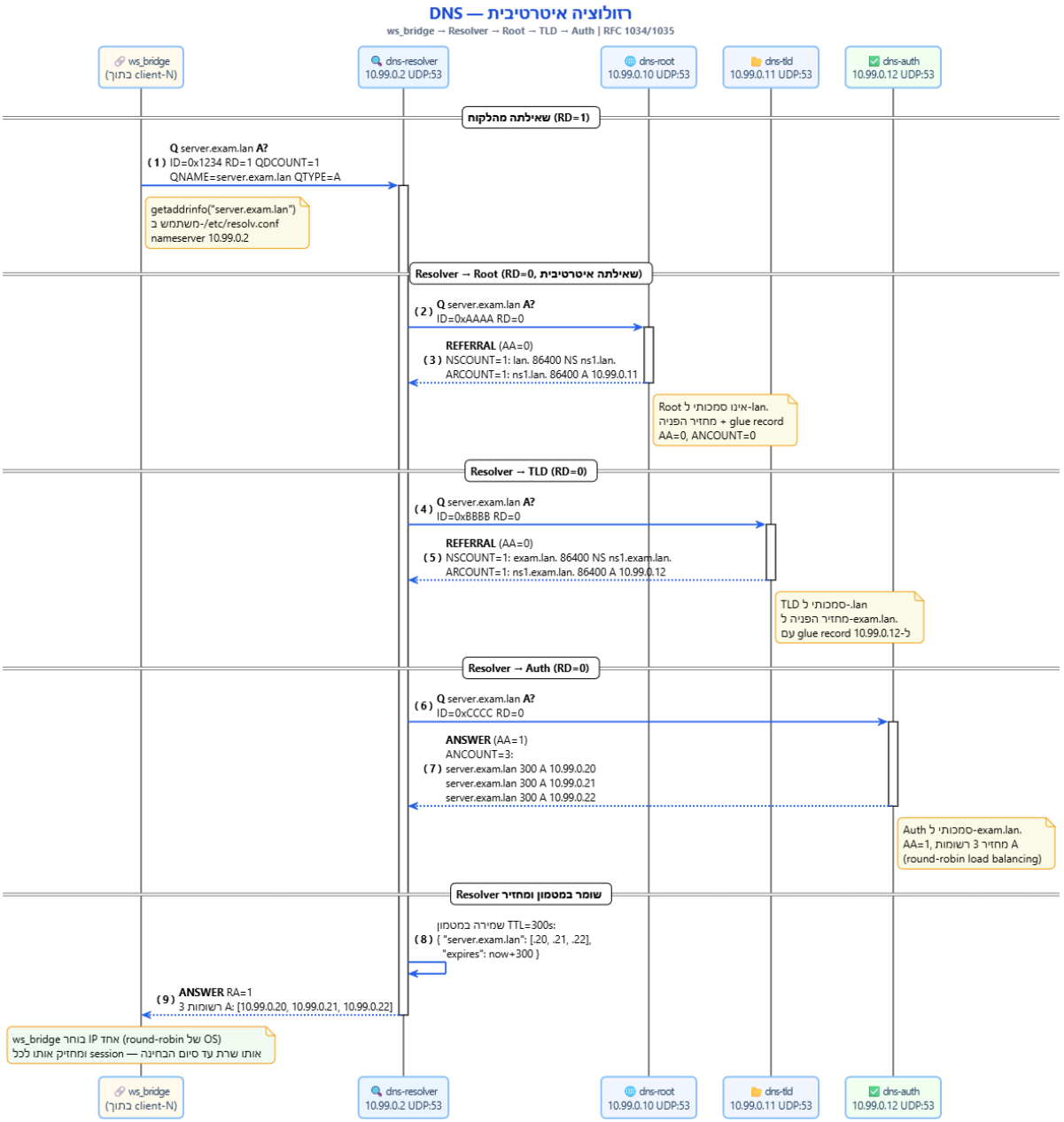


2. DNS Local Server

לאחר קבלת כתובת IP, הלקוח פונה לשרת ה-DNS כדי לתרגם את השם של שרת האפליקציה לכתובת שלה ברשת. אנחנו עושים זאת לפי ההיררכיה השלמה של DNS

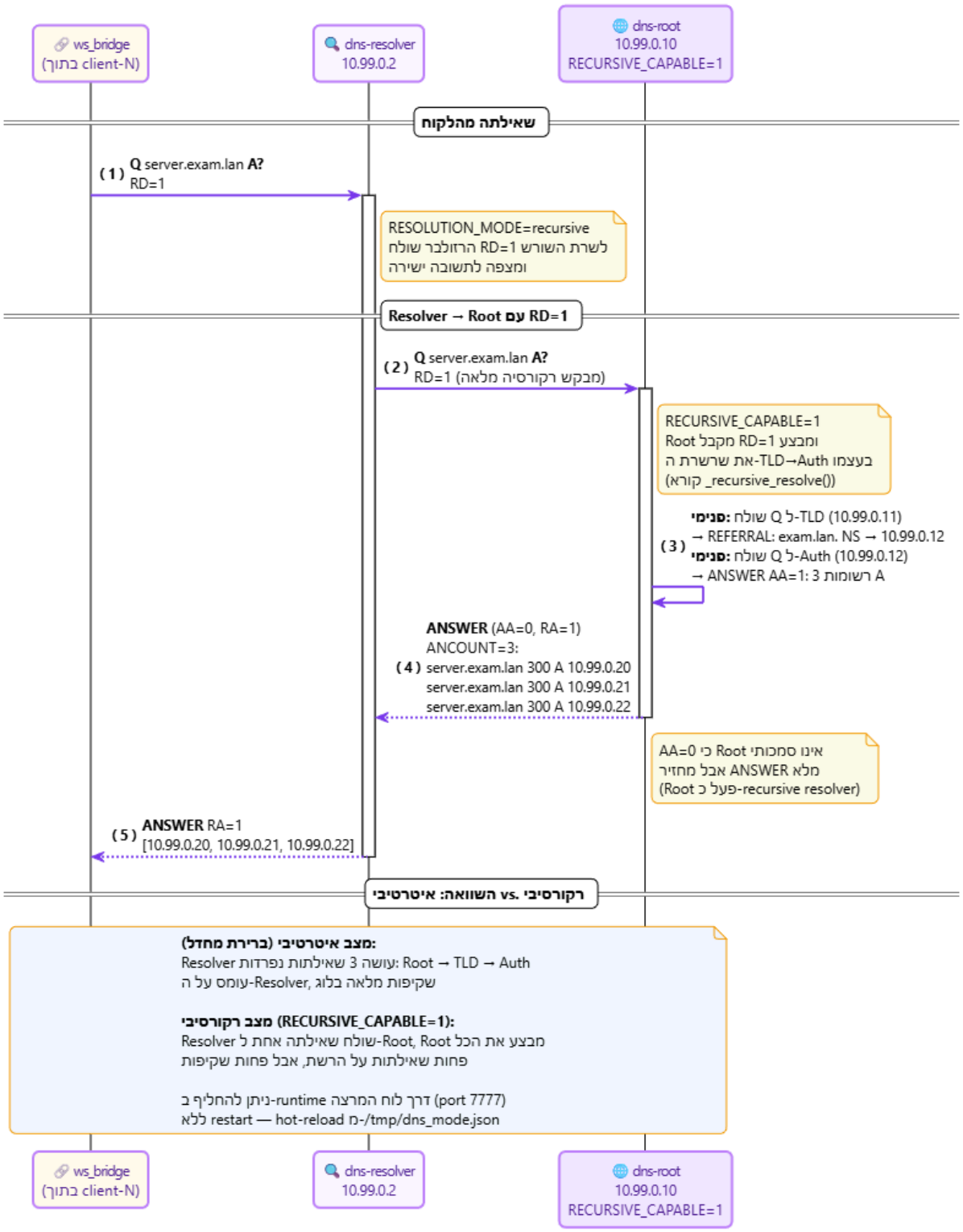
ROOT-> TLD->AUTH - client

אנחנו עושים את זה בשני הדרכים תלוי אפשרות - גם רקורסיבי וגם איטראטיבי (יש כפתור בדאשבורד שמראה את שניהם)



DNS — רזולוציה רקורסיבית

RECURSIVE_CAPABLE=1 | RESOLUTION_MODE=recursive | toggle via prof dashboard



3. Application Server

שרת האפליקציה אחראי על ניהול ושליחת המבחן למשתמשים.
המבחן מוצג על ידי קובץ JSON המכיל את השאלות.
השרת תומך בשתי שיטות תקשורת מול הלקוחות:

- **TCP**
- **RUDP**

אולם כדי לקבל את זמן השליחה הקטן ביותר עדיף להשתמש ב-RUDP וזה במה שהשתמשנו בהשוואה לאלגוריתם החיצוני.

מבנה לוגי של שרת האפליקציה

Docker

בשביל שנוכל להריץ את כל היוזרים בצורה יעילה וקלה (עשינו הרצות עם 150+ יוזרים במקביל)
השתמשנו בתוכנה DOCKER שהיא נותנת לנו את האפשרות להריץ כמה מכונות וירטואליות שאנחנו רוצים בצורה אוטומטית. כל איבר בתוכנה שלנו הוא קונטיינר משל עצמו (DHCP,DNS,application,users)

ניהול עומסים וניתוב ללקוחות

כדי להתמודד עם מספר רב של משתמשים ולעשות traffic control, אנחנו משתמשים בשני מנגנונים שונים לניהול עומסים. הראשון הוא DNS Round-Robin. מאחר שישנם שלושה קונטיינרים של שרתי RUDP (בכתובות 10.99.0.20, 10.99.0.21, 10.99.0.22), שרת ה-DNS האוטוריטטיבי מגדיר שלוש רשומות A שונות תחת השם הבודד server.exam.lan. כאשר ה-bridge בצד הלקוח קורא ל"(getaddrinfo("server.exam.lan, הDNS מחזיר את כל שלוש הרשומות ומערכת ההפעלה בוחרת אחת מהן לפי סדר round-robin. כך נוצר פיזור טבעי של החיבורים - סטודנטים שונים מתחברים לשרתים שונים, ללא צורך במרכז תיאום נוסף. המנגנון השני הוא reverse proxy מקומי עם NGINX. בתוך כל קונטיינר לקוח, NGINX פועל כreverse proxy פנימי שמקבל בקשות WebSocket מהדפדפן ומנתב אותן לbridge המתאים לפי נתיב הבקשה: /ws מגיע לbridge ה-/ws-tcp-sync, RUDP (port 8081) לbridge ה-TCP עם סינכרון (port 8082), ו/ws-tcp-nosync ל-bridge ה-TCP ללא סינכרון (port 8083).

על מנת לעשות את התיאום זמנים וקבלת ההחלטות של איזו בקשה להעביר קודם, הגדרנו Manager (בקוד מופיע כ-ExamSendCoordinator) שרץ בתוך כל שרת. הmanager משתמש בקובץ משותף (app/shared/start_at.txt/) שנגיש לכל מופעי השרתים כך כולם עובדים מול אותו זמן התחלה מוגדר.
החלק של הניהול תזמון השליחה ועדיפות הבקשות כתוב בהמשך.

תהליך העבודה במערכת

שלב 1 - התחברות לקוח לרשת וקבלת IP

הלקוח מתחבר לשרת DHCP ומקבל כתובת IP לפי DORA עם סיום התהליך אוטומטית מופעל NGINX וbridge (הסקריפט שקובע אם נשתמש ב TCP+ no sync, TCP+sync,RUDP).

שלב 2 - גילוי השירות

הברידג' פונה לשרת DNS המקומי (עובד גם איטרטיבית וגם רקורסיבית תלוי דגל) ומקבל שלוש רשומות ונבחר אחת לפי round robin ואז נוצר פיזור. (יש את התרשימים למעלה)

שלב 3 - חיבור לשרת האפליקציה

הלקוח מתחבר לאתר מציג מסך בחירת מצב (RUDP Sync / TCP Sync / TCP No-Sync). לאחר הבחירה נפתח חיבור WebSocket, nginx מנתב אותו לbridge המתאים, והbridge מבצע לחיצת יד לפי הפרוטוקול הנבחר. ומעדכן את הManager לגבי זמני חיבור. ניתן לראות את לחיצת היד של RUDP של הbridge והשרת:

RUDP — 3-Way Handshake

Header: seq(4) ack(4) flags(1) window(1) len(2) crc16(2) = 14 bytes

ws_bridge (client-N) | exam-server 10.99.0.2x UDP:9000

RUDP Header Format (14 bytes):

seq (4B)	ack (4B)	flags (1B)	window (1B)	len (2B)	crc16 (2B)

Flags: SYN=0x01 ACK=0x02 FIN=0x04 DATA=0x08 RST=0x10 PING=0x20 MSG_END=0x40

חיבור + MSS Negotiation

t1 = time.monotonic()
(לחישוב handshake_rtt_ms)

(1) **[SYN]**
seq=0 ack=0 flags=0x01
window=64 len=2
payload=0x04B0 (1200 — MSS)
crc16=<checksum>

(2) **[SYN+ACK]**
seq=0 ack=1 flags=0x03
window=64 len=2
payload=0x04B0 (1200 — MSS)
crc16=<checksum>

חדש RudpSocket שרת מקצה
מאתחל send_buf, recv_buf
window=64 = RECV_BUF_SIZE (64 פקטות)
MSS negotiation:
remote_mss = int.from_bytes(payload[:2])
conn._mss = min(1200, remote_mss) = **1200**

(3) **[ACK]**
seq=1 ack=1 flags=0x02
window=64 len=0
crc16=<checksum>

MSS negotiation (client side):
remote_mss = int.from_bytes(syn_ack.payload[:2])
conn._mss = min(1200, remote_mss) = **1200**
כל DATA chunk: payload ≤ 1200 bytes

מדידת RTT ה של-Handshake

t2 = time.monotonic()
handshake_rtt_ms = (t2 − t1) × 1000
נשמר ב-RudpSocket
ומוזרק ל-hello בהמשך

מוכן Sliding Window — מצב חיבור

חיבור נוצר בהצלחה ✓
cwnd = 1 (Slow Start)
ssthresh = 32
effective_window = min(cwnd, peer_window, WINDOW_CAP=32)

Timeout לכל packet: 500ms, max 5 retries → RudpTimeout
5 לאחר retries: RST + סגירת חיבור

סגירת חיבור (עתידית)

(4) **[FIN]**
seq=N flags=0x04

(5) **[FIN+ACK]**
ack=N+1 flags=0x06

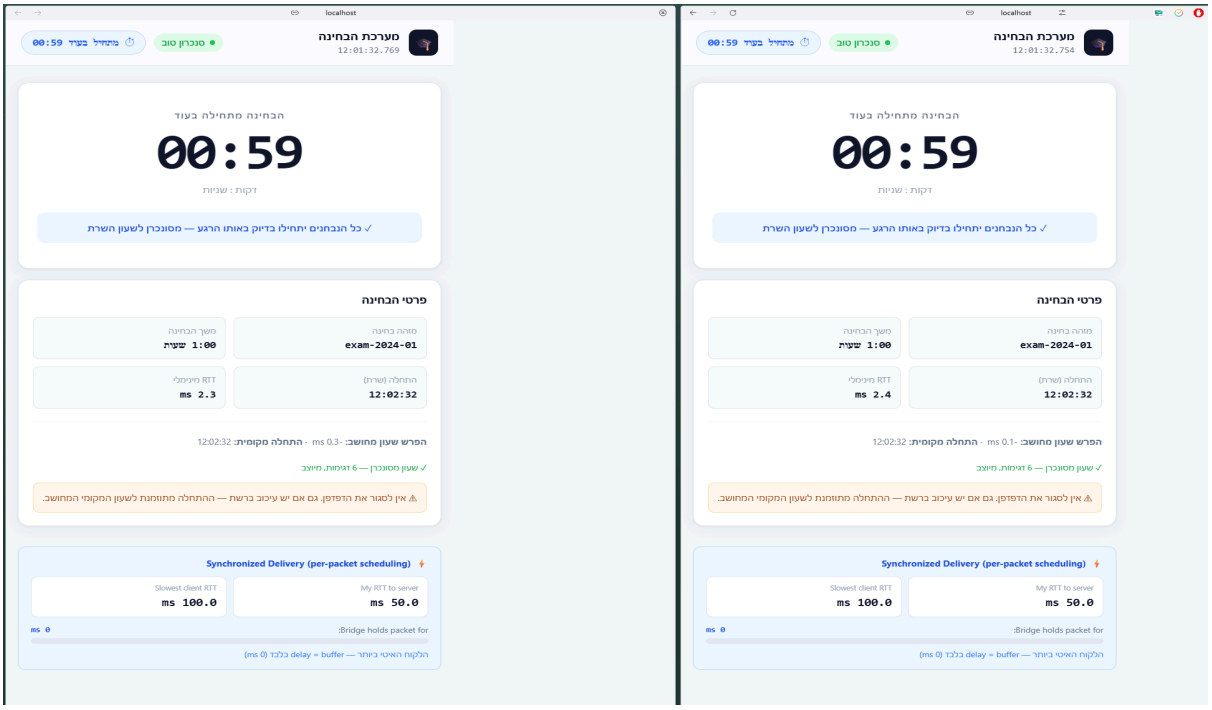
מסיים חיבור בצורה מסודרת FIN
(לדוגמה: סגירת דפדפן → WebSocket close → bridge close → RUDP FIN)

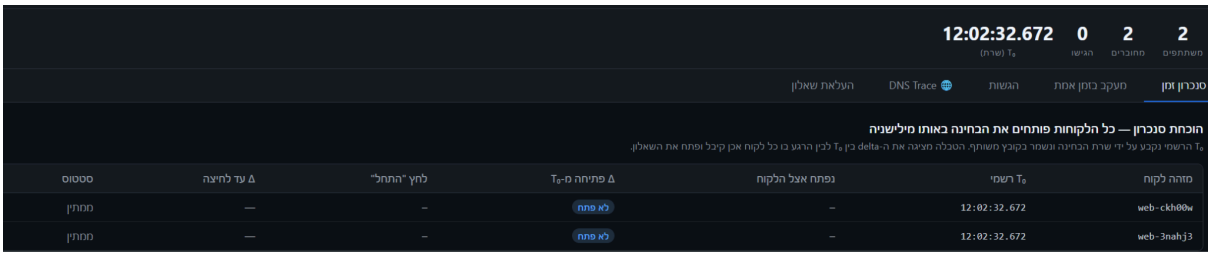
ws_bridge (client-N) | exam-server 10.99.0.2x UDP:9000

שלב 4 - טעינת מבחן

מפעיל המערכת מעלה מבחן בפורמט JSON, הכולל את כל שאלות המבחן.

הלקוח שולח schedule_req ומקבל את זמן ההתחלה המשותף (start_at_ms), שמאוחסן ב-volume/— app/shared/start_at.txt משותף לכל מופעי השרת. לאחר מכן exam_req מחזיר את תוכן הבחינה המלא. ניתן לראות את הפרש השעות המחושב





שלב 5 - מדידות רשת

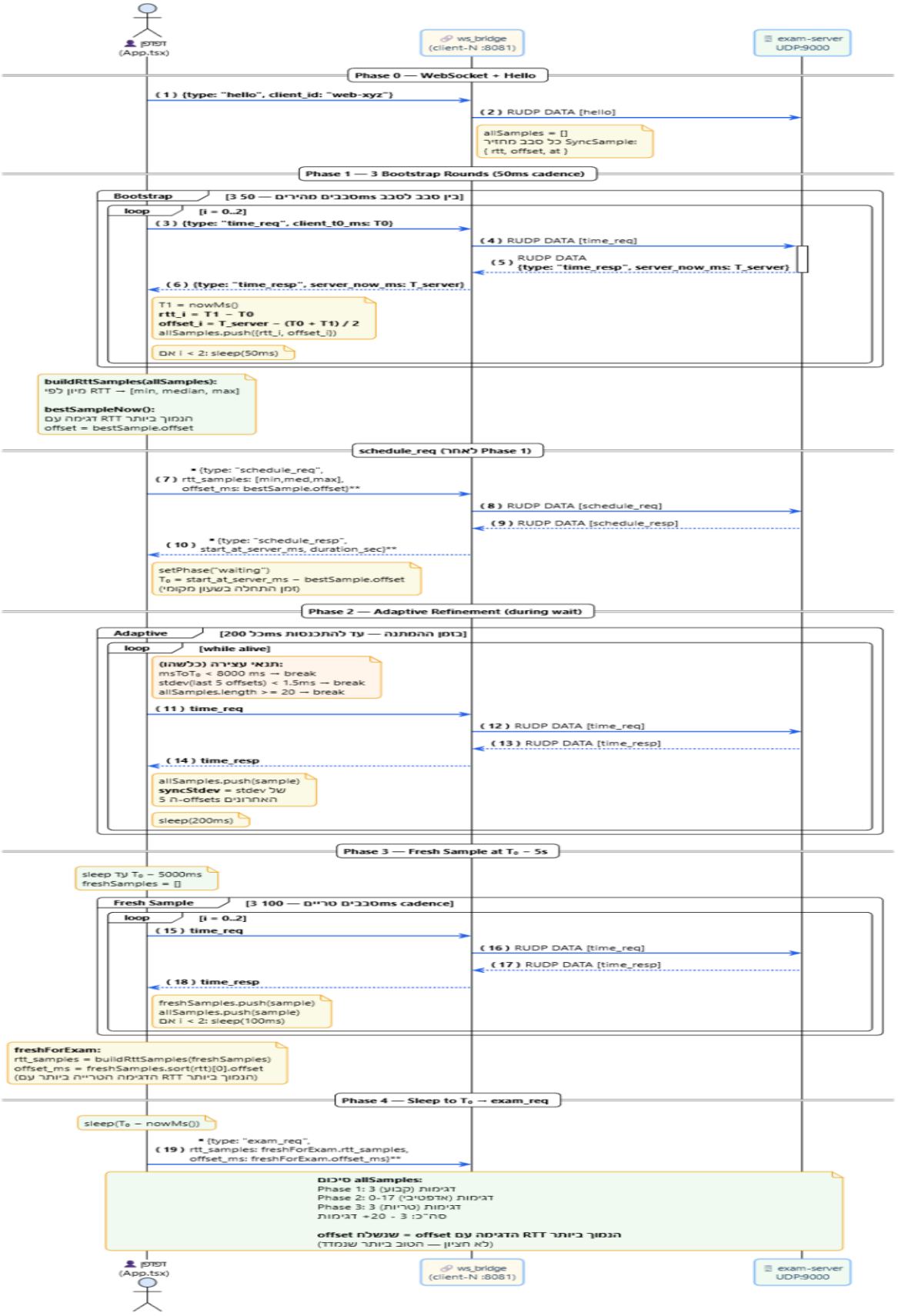
לפני תחילת ההפצה המערכת מבצעת מדידות רשת לכל המשתמשים, ובפרט:

- RTT - הלקוח מבצע 12 סבבי time_req/time_resp לפי נוסחת NTP, ומחשב RTT ו-clock offset. הוא מסכם את המדידות ל-rtt_min, rtt_med, rtt_max
- Jitter (מחושב מהRTT)
- גודל חבילה מקסימלי שמתאים לכולם

המטרה היא למצוא פרמטרי שליחה שיאפשרו העברה הוגנת לכלל המשתמשים, ולא רק למשתמש המהיר ביותר. בחלק הזה המערכת מוצאת את היוזרים שהולכים לעבוד איתה, אם בשלב הזה יוזר מתנתק המערכת ממשיכה בלעדיו.

RTT Sync — Adaptive Clock Synchronisation

Phase 1: bootstrap (3) → schedule_req · Phase 2: adaptive refinement · Phase 3: fresh sample (3) → exam_req



שלב 6 - תזמון הוגן של שליחה

במהלך ההפצה המשתמשים ממוינים לפי RTT. (החישוב המלא כאן)
השליחה מתחילה למשתמש בעל RTT גבוה יותר, ולאחר מכן למשתמשים הבאים, עם delay מחושב בין שליחה לשליחה. (זה נעשה על ידי כך שהmanager אוסף מדידות מכלל הלקוחות ומחשב לכל אחד את זמן ההגעה החד-כיווני: $D_i = (RTT_{med} + j_{rtt})/2$. השרת שולח לכל לקוח ב $T_{send,i} = T_{arr} - D_i$

הלקוח הרחוק יותר נשלח קודם. ה-bridge משפר את זה עם מנגנון עיקוב של max_rtt/2 + 5ms. התוצאה: כל הדפדפנים מקבלים את הבחינה בסטייה של ±5ms.

הרעיון הוא לפצות על הבדלי זמני ההגעה:

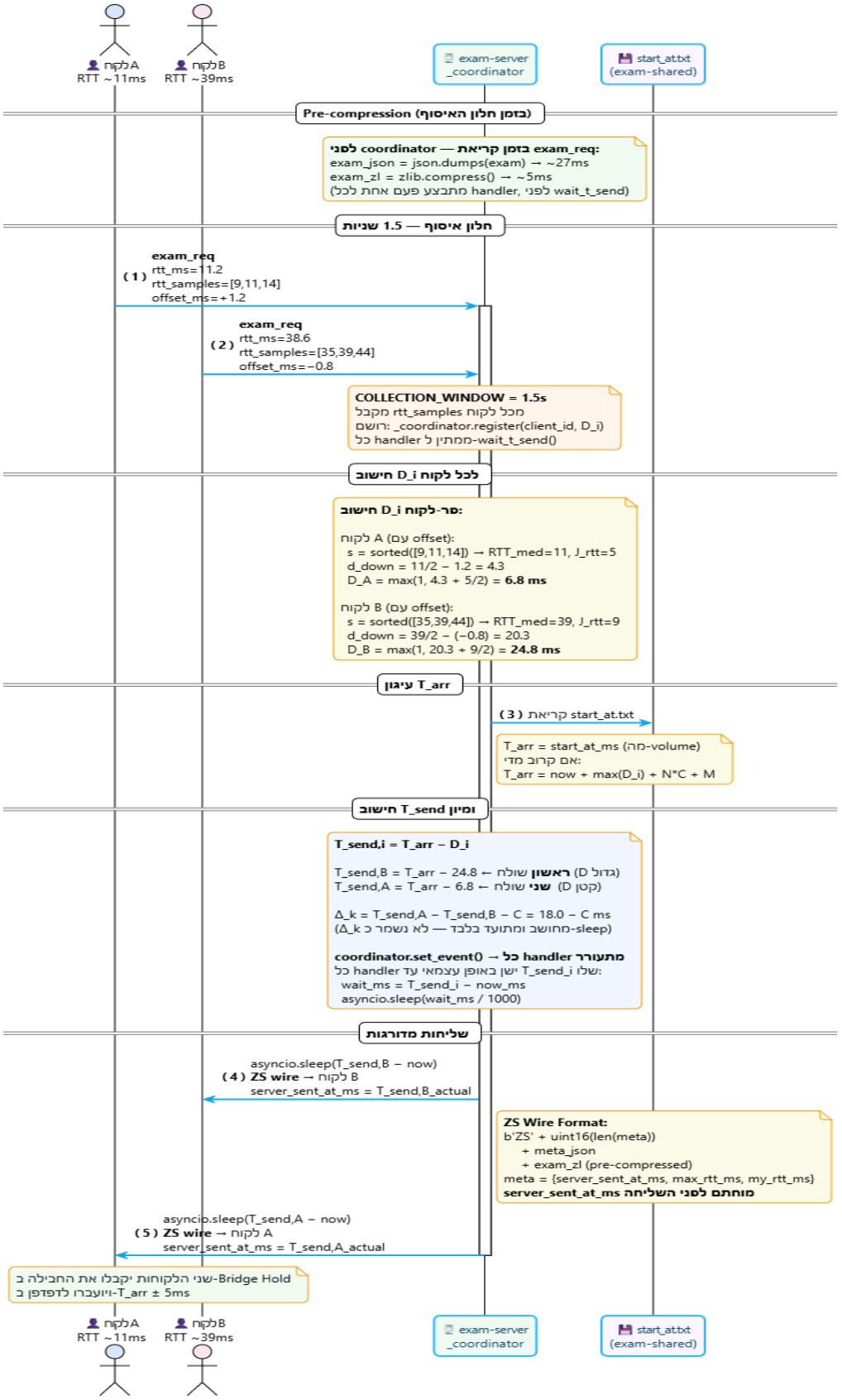
- משתמש עם RTT גבוה יקבל את החבילה מוקדם יותר מצד השרת
- משתמש עם RTT נמוך יקבל אותה מעט מאוחר יותר

כך, בפועל, החבילה אמורה להגיע לכלל המשתמשים כמעט באותו הזמן.

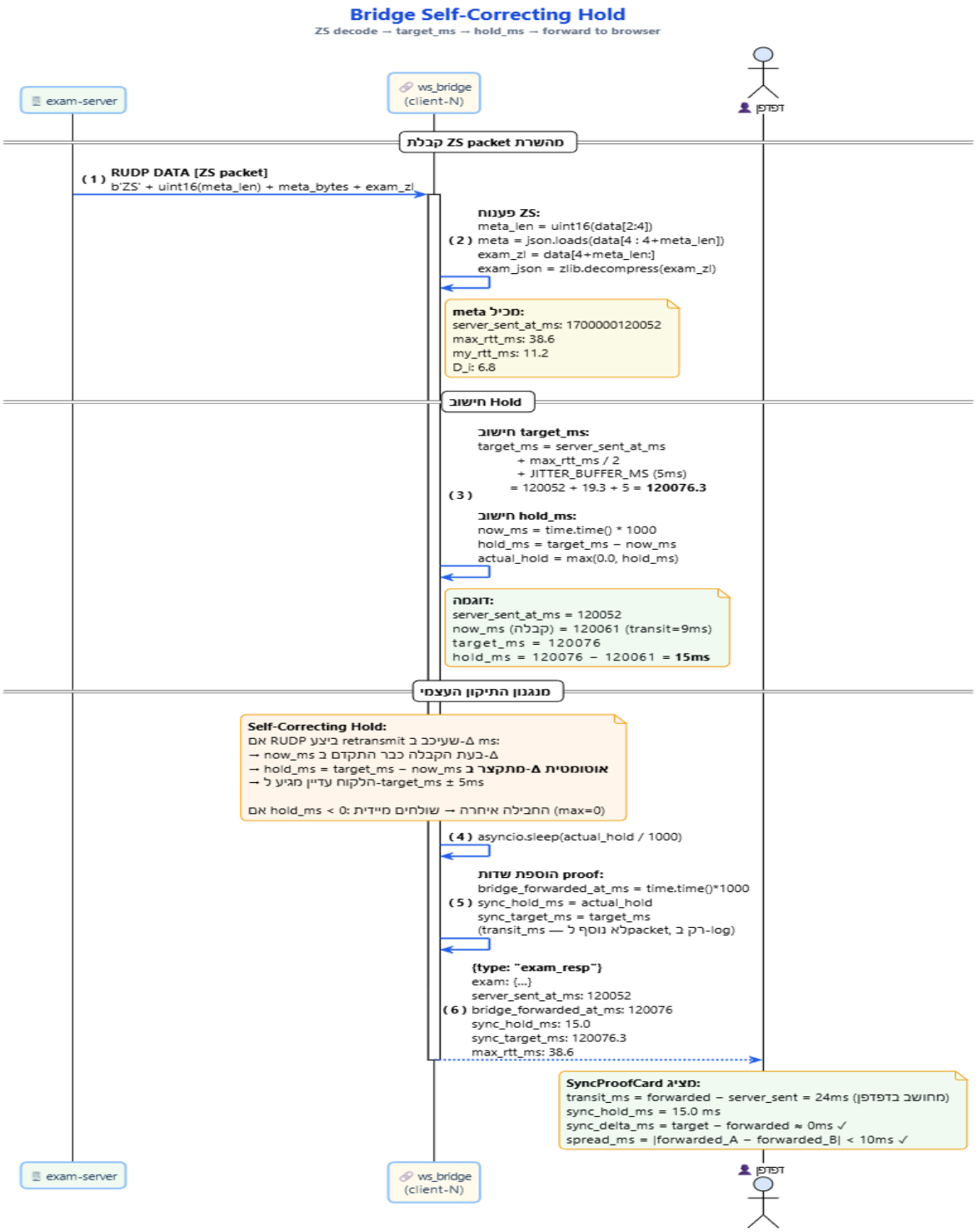
הmanager:

ExamSendCoordinator — שליחה מדורגת

איסוף D_i → חישוב T_send → stagger → ZS wire format

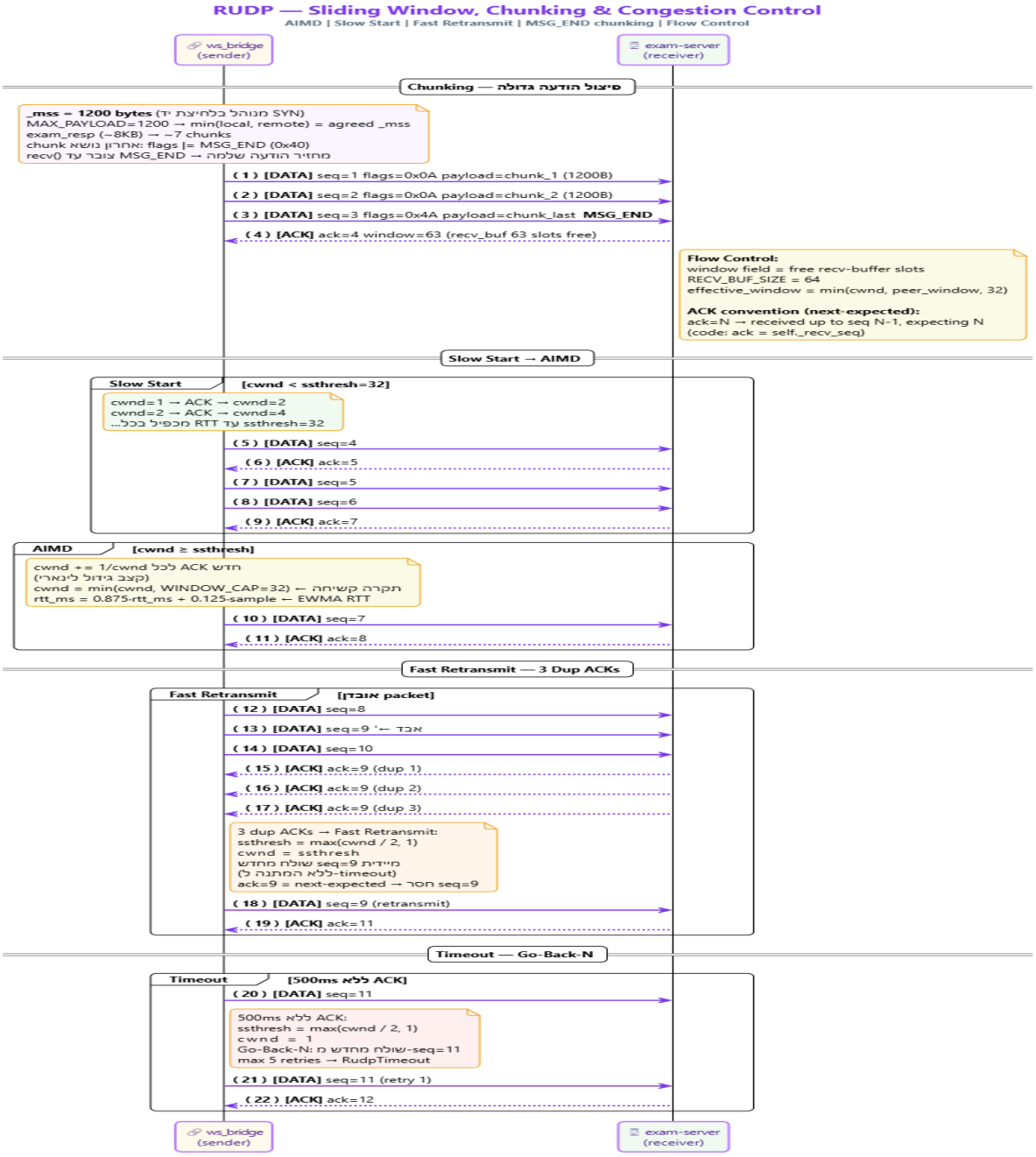


:Bridge


Bridge Self-Correcting Hold
ZS decode → target_ms → hold_ms → forward to browser
exam-server
ws_bridge (client-N)
דפדפן
מהשרת ZS packet קבלת
(1) RUDP DATA [ZS packet]
b'ZS' + uint16(meta_len) + meta_bytes + exam_zl
פענוח ZS:
meta_len = uint16(data[2:4])
(2) meta = json.loads(data[4 : 4+meta_len])
exam_zl = data[4+meta_len:]
exam_json = zlib.decompress(exam_zl)
מכיל meta:
server_sent_at_ms: 1700000120052
max_rtt_ms: 38.6
my_rtt_ms: 11.2
D_i: 6.8
חישוב Hold
חישוב target_ms:
target_ms = server_sent_at_ms
+ max_rtt_ms / 2
+ JITTER_BUFFER_MS (5ms)
= 120052 + 19.3 + 5 = 120076.3
(3)
חישוב hold_ms:
now_ms = time.time() * 1000
hold_ms = target_ms − now_ms
actual_hold = max(0.0, hold_ms)
דוגמה:
server_sent_at_ms = 120052
now_ms (קבלה) = 120061 (transit=9ms)
target_ms = 120076
hold_ms = 120076 − 120061 = 15ms
מנגנון התיקון העצמי
Self-Correcting Hold:
אם RUDP ביצע retransmit ב-שעיכב Δ ms:
→ now_ms ב בעת הקבלה כבר התקדם ב-Δ
→ hold_ms = target_ms − now_ms ב אוטומטית Δ-מתקצר
→ target_ms ± 5ms-הלקוח עדיין מגיע ל
אם hold_ms < 0: שולחים מיידית → החבילה איחרה (max=0)
(4) asyncio.sleep(actual_hold / 1000)
הוספת שדות proof:
bridge_forwarded_at_ms = time.time()*1000
(5) sync_hold_ms = actual_hold
sync_target_ms = target_ms
(transit_ms — לא נוסף ל packet, רק ב-log)
{type: "exam_resp"}
exam: {...}
server_sent_at_ms: 120052
(6) bridge_forwarded_at_ms: 120076
sync_hold_ms: 15.0
sync_target_ms: 120076.3
max_rtt_ms: 38.6
SyncProofCard מציג:
transit_ms = forwarded − server_sent = 24ms (מחושב בדפדפן)
sync_hold_ms = 15.0 ms
sync_delta_ms = target − forwarded ≈ 0ms ✓
spread_ms = |forwarded_A − forwarded_B| < 10ms ✓
exam-server
ws_bridge (client-N)
דפדפן

שלב 7 - שיפור דינמי של המדידות

בזמן ההמתנה לאישורי קבלה, המערכת אוספת דגימות נוספות מתוך התעבורה עצמה, וכך משפרת את חישוב ה־RTT וה־Jitter לאורך זמן.
לכן המערכת אינה מסתמכת רק על מדידה חד־פעמית בתחילת הריצה, אלא מתאימה את עצמה דינמית בנוסף לשימוש בslow start,AIMD בדאשבורד של הדמו ניתן להוסיף delay,jitter,loss ולראות את ההתמודדות



שלב 8 - טיפול בהתנתקות

אם משתמש מתנתק:

- כל שינוי בתשובות נשמר מיד בשרת (/answers_save -> /app/answers/). סיום תקין מתבצע בלחיצת יד FIN. התנתקות לא צפויה מזוהה לאחר 5 ניסיונות retransmit כושלים (2.5 שניות) - הbridge סוגר את ה-WebSocket, והמשתמש יכול להתחבר מחדש וימשיך מהנקודה האחרונה שנשמרה.
- אם המשתמש חוזר בזמן, התקשורת ממשיכה
- אם לא, המשתמש מוסר מהמערכת וההפצה ממשיכה לשאר המשתמשים

כיצד המערכת שומרת על קבלת חבילות תקינה

כדי להתמודד עם אובדן חבילות ברשת אנחנו מיישמים כמה מנגנוני אמינות

בדיקת תקינות (CRC-16): (במטרה לחקות צ'קסם) כל חבילת RUDP נושאת סכום ביקורת CRC-16/CCITT-FALSE המחושב על פני הכותרת כולה וה-payload. הצד המקבל מחשב מחדש את ה-CRC ומשווה לערך שבכותרת - חבילה עם אי-התאמה נפסלת ונזרקת. מנגנון ה-timeout שיתואר להלן יגרום לשידורה החוזר.

ניהול MSS במהלך לחיצת היד: לפני שמתחיל כל שידור נתונים, כל צד מכריז על ה-MSS (Maximum Segment Size) שלו - הגודל המרבי של payload שהוא מסוגל לקבל בחבילה אחת. ערך זה מועבר כ2 בייטים big-endian ב-payload של חבילת ה-SYN (מהלקוח לשרת) וב-SYN-ACK (מהשרת ללקוח). הצד המקבל מחשב את ה-MSS המוסכם לפי min(local, remote) ומאחסן אותו ב-self._mss. כל פיצול נתונים לאחר מכן מתבסס על ערך מוסכם זה ולא על קבוע סטטי.

אישורי קבלה מצטברים (Cumulative ACK): הצד המקבל שולח ACK לאחר כל חבילת DATA, המציין את מספר הרצף הגבוה ביותר שהתקבל ברצף רציף. אם הגיעו חבילות עם מספרי רצף 1, 2, 4 ה-ACK יציין 3, כיוון שחבילה מספר 3 עדיין חסרה. חבילה 4 מוחזקת בינתיים בבאפר חבילות מחוץ לסדר ותועבר לשכבת האפליקציה ברגע שהפער יתמלא.

תזמון השליחה:

כדי להתמודד עם זמני השהיה שונים בין המשתמשים, המערכת מבצעת תחילה סנכרון שעונים בין הלקוח לשרת. בשלב לחיצת היד, כל צד מכריז על גודל החבילה המרבי שהוא מסוגל לקבל (MSS), והצדדים מסכימים על min(local, remote) ומשתמשים בו לכל פיצול החבילות. לאחר מכן הלקוח מבצע 12 סבבי time_req/time_resp לפי נוסחת NTP, ומכל סבב נגזרים rtt_i והיסט שעון. בסיום נשמרים שלושה ערכי RTT מייצגים — מינימום, חציון ומקסימום — והיסט השעון המחושב מסבבי ה-NTP.

ערכים אלו נשלחים לשרת בהודעת schedule_req, ומזינים את חישוב D_i

אומדן זמן ההגעה החד-כיווני של כל לקוח, הכולל RTT ו-jitter:

$$s = sorted([rtt_{min}, rtt_{med}, rtt_{max}])$$

$$RTT_{med} = s[1] \ , J_{rtt} = s[2] - s[0]$$

אם יש offset

$$D_i = max(1, RTT_{med}/2 - offset + J_{rtt}/2) :$$

אחרת: $D_i = (RTT_{med} + J_{rtt})/2$

לקוח בעל jitter גבוה או RTT גבוה יקבל D_i גדול יותר ובהתאם, החבילה תישלח אליו מוקדם יותר כדי לפצות על העיכוב. ה-Manager אוסף את מדידות כלל הלקוחות בחלון זמן של 1.5 שניות, ולאחר סגירתו מחשב לכל לקוח את זמן השליחה:

T_arr = start_at_ms (fallback: t_now + max(D_i) + N·C + M או)

T_send,i = T_arr − D_i

הקואורדינטור ממיין את הלקוחות לפי T_send עולה — הרחוק ביותר נשלח ראשון — ומשחרר את כל ה-handlers בו-זמנית. כל handler ישן באופן עצמאי עד לזמן הפרטי שלו ואז שולח את תוכן הבחינה עם חותמת server_sent_at_ms. ערך T_arr נקרא מקובץ משותף לכל מופעי השרת (app/shared/start_at.txt/), כך שגם לקוחות שהתחברו לשרתים שונים מכוּוָנים אל אותו זמן יעד.

בצד ה-bridge מופעל מנגנון תיקון עצמי: לאחר קבלת החבילה, ה-bridge מחשב את הזמן שנותר עד לרגע המיועד לשליחה לדפדפן:

$$target_ms = server_sent_at_ms + max_rtt/2 + 5ms$$

$$hold_ms = max(0, target_ms - now_ms())$$

אם שידור חוזר של RUDP עיכב את החבילה, now_ms() כבר התקדם בהתאם

hold_ms מתקצר אוטומטית ו-target_ms נשמר קבוע. כך, ללא תלות ב-RTT האישי של כל לקוח, כלל הדפדפנים מקבלים את הבחינה בסטייה של ±5ms בלבד:

זמן הגעה לדפדפן = T_send,i + D_i + hold_ms ≈ T_arr ± 5ms

מקרי קצה וטיפול בשגיאות

המערכת מתייחסת גם למקרי קצה, בהתאם לדרישת המטלה לטפל בבאגים וב־edge cases:

- משתמש שמתנתק באמצע ההפצה
- משתמש שלא שולח ACK בזמן
- הבדלים גדולים במיוחד ב־RTT בין משתמשים ודיליי מלאכותי
- גודל חבילה שאינו מתאים לכלל הלקוחות
- עבודה תחת המתנות ואיבודי חבילות לצורך בדיקות

סיכום

המערכת מממשת תהליך רשת מלא הכולל DHCP, DNS, ושרת האפליקציה.

הפתרון והמימוש שהתמקדנו בו זה שליחת שאלות מבחן לכמות גדולה של משתמשים בצורה מסונכרנת והוגנות לכל פאקטה. תוך כדי שימוש ב-Managers לתיאום תזמונים,קבלת החלטות סדר שליחה ומדידת jitter,delay , RTT ומדדים נוספים על מנת לקבוע זמן הגעה אחיד כמה שיותר מדויק לכל אחד מהמשתמשים.

היתרון המרכזי של המערכת הוא שלא מדובר רק בהעברת קובץ, אלא בהעברה הוגנת, מסונכרנת ודינמית, יותר מהירה מאלגוריתמים קיימים וגם קונספט שאפשר לממש אותו בהרבה סוגי פרויקטים עם שינוי קוד מינימלי.tn,